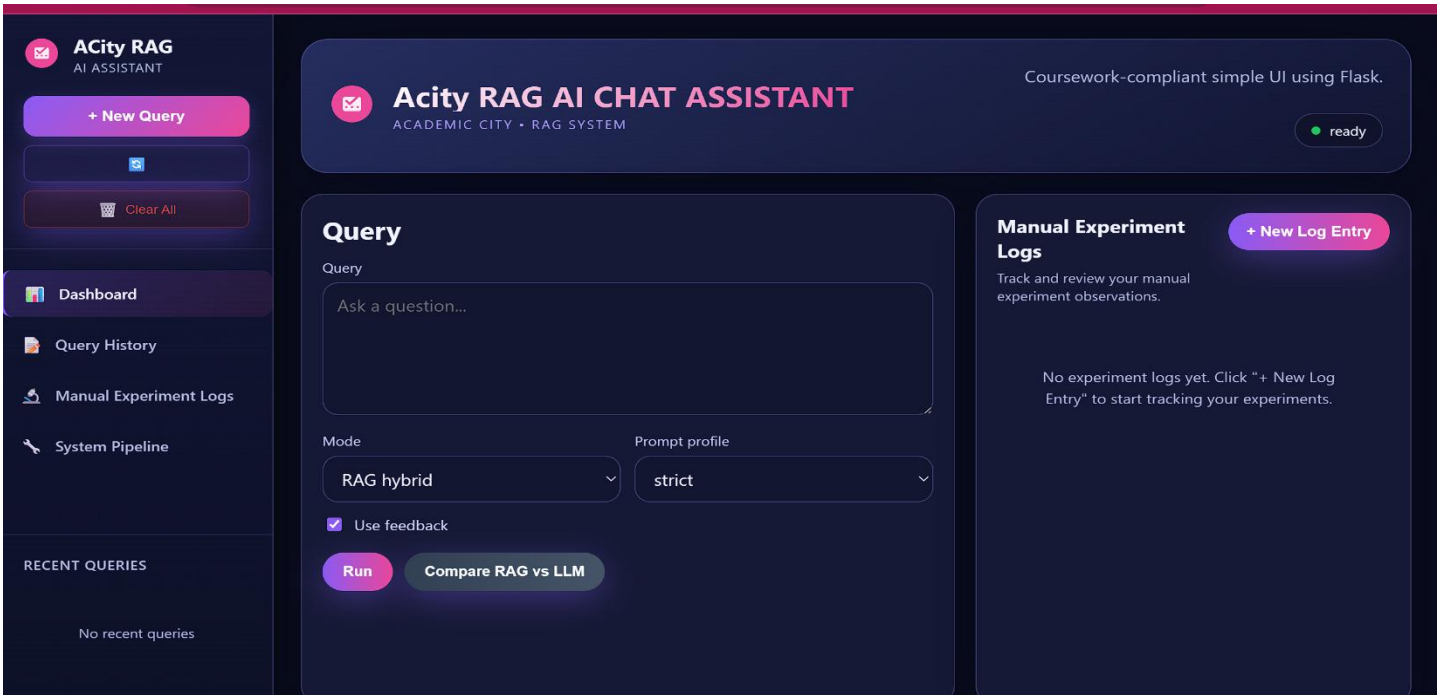
ACADEMIC CITY UNIVERSITY

FACULTY OF COMPUTATIONAL SCIENCES AND INFORMATICS
END OF SECOND SEMESTER EXAMINATION – 2026
CS4241-Introduction to Artificial Intelligence
Lecturer: Godwin N. Danso

AI_10012300017
Name: José Maria Kombila

Project based Title : ACity RAG AI Chat Assistant



1. Introduction

This document explains how I built the Acity RAG AI Chat Assistant for my Introduction to AI exam Project Based. The app helps users ask questions about old Ghana's election results, the 2025 Budget Statement and Economic Policy. It gets smart answers from a computer system.

1.1 What is RAG?

RAG stands for Retrieval-Augmented Generation. In simple words:

- A user types a question.
- The system searches through real documents to find the most useful parts.
- It then gives those parts to an AI, which writes a clear answer.

This is better than just asking an AI directly, because the AI uses real, specific information instead of guessing.

1.2 Project Goal

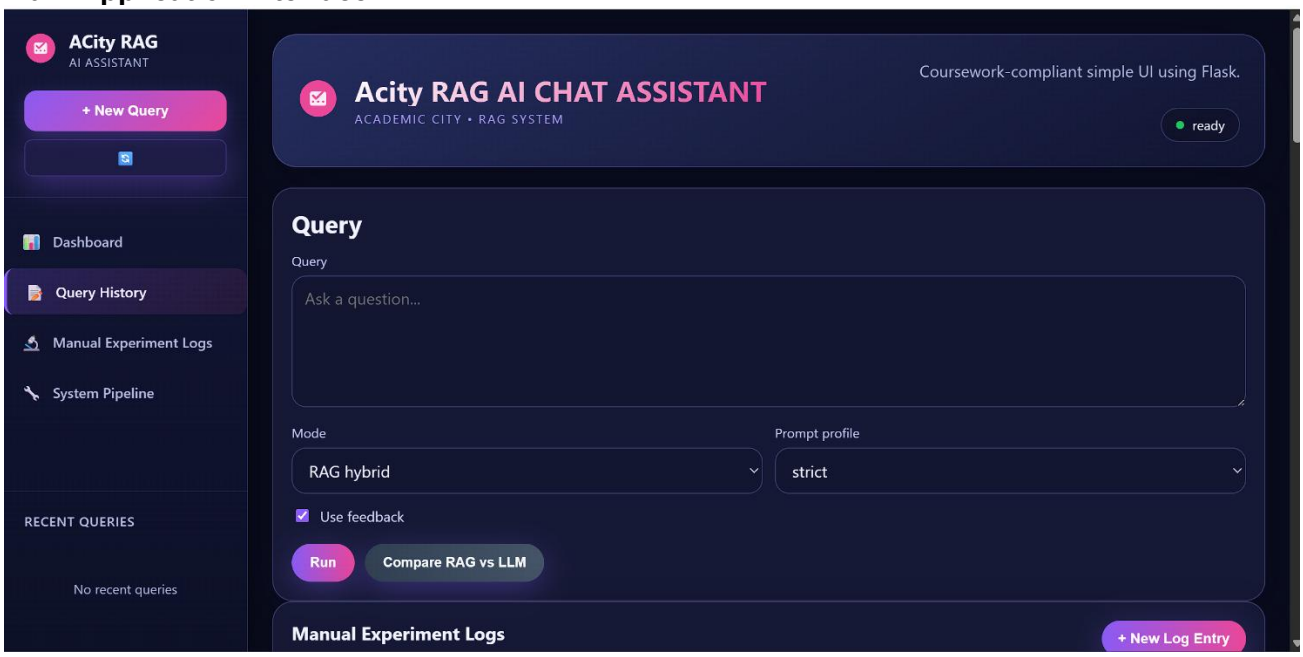
Build a chat app that can answer questions about:

- Ghana Election Results (CSV dataset from GitHub)
- Ghana 2025 Budget Statement (PDF from the Ministry of Finance website)

The system was built WITHOUT using ready-made AI frameworks like LangChain or LlamaIndex. Every part was coded manually.

1.3 Application Interface Overview

Main Application Interface



Query Example

Query

How many votes did Rawlings get?

Mode: RAG hybrid

Prompt profile: strict

Use feedback

Run | Compare RAG vs LLM

Logs

Track and review your manual experiment observations.

#1 2026-04-24 20:07:12 SUCCESS

How many votes did Rawlings get? →

rag_hybrid

Answer

According to the Ghana 1992 presidential election results in the Western Region, Jerry John Rawlings from NDC party received 239477 votes.

Retrieved chunks

csv:1992:Western Region:9 d=0.413 | b=8.795 | h=1.000

Ghana presidential election data: Ghana 1992 presidential election results: Jerry John Rawlings from NDC party received 239477 votes (60.74% percentage) in Western Region region. Election candidate Jerry John Rawlings performance in Western Region with vote count 239477. Ghana 1992 presidential election results: Albert Adu Boahen from NPP party received 89800 votes (22.78% percentage) in Western R...

csv:1992:Central Region:2 d=0.412 | b=8.795 | h=0.997

Ghana presidential election data: Ghana 1992 presidential election results: Jerry John Rawlings from NDC party received 222092 votes (66.49% percentage) in Central Region region. Election candidate Jerry John Rawlings performance in Central Region with vote count 222092. Ghana 1992 presidential election results: Albert Adu Boahen from NPP party received 86683 votes (25.95% percentage) in Central R...

csv:1992:Volta Region:8 d=0.404 | b=8.795 | h=0.986

Ghana presidential election data: Ghana 1992 presidential election results: Jerry John Rawlings from NDC party received 446365 votes (93.24% percentage) in Volta Region region. Election candidate Jerry John Rawlings performance in Volta Region with vote count 446365. Ghana 1992 presidential election results: Albert Adu Boahen from NPP party received 17295 votes (3.61% percentage) in Volta Region r...

csv:1992:Upper East Region:6 d=0.406 | b=8.714 | h=0.982

Ghana presidential election data: Ghana 1992 presidential election results: Jerry John Rawlings from NDC party received 108999 votes (53.97% percentage) in Upper East Region region. Election candidate Jerry John Rawlings performance in Upper East Region with vote count 108999. Ghana 1992 presidential election results: Albert Adu Boahen from NPP party received 21164 votes (10.48% percentage) in Upp...

2. Part A - Data Engineering and Preparation

2.1 Data Sources

Two datasets were used for this project:

Dataset	Description
Ghana Election Results	A CSV file with voting numbers by region and candidate from the 2024 Ghana election.
Ghana 2025 Budget	A large PDF from the Ministry of Finance with Ghana's economic plan for 2025.

2.2 Data Cleaning

Before using the data, I had to clean it. Here is what I did:

For the Election CSV file:

- Removed empty rows and columns.
- Fixed spelling mistakes in region names.
- Made sure all numbers (votes) were in the correct format.
- Removed duplicate rows.

For the Budget PDF file:

- Extracted all the text from the PDF pages.
- Removed page headers, footers, and page numbers that repeated on every page.
- Fixed broken sentences that were split across two lines.
- Removed special characters that did not add meaning.

2.3 Chunking Strategy

Chunking means cutting the documents into small pieces so the computer can search them easily.

Setting	Value Used
Chunk Size	500 words per chunk
Overlap	50 words shared between chunks
Method	Sliding window (move forward 450 words at a time)

Why 500 words?

- Small enough for the search system to find specific answers.
- Big enough to keep the full meaning of a paragraph together.
- Not too large, which would slow down the system.

Why 50 words of overlap?

- Some sentences cross the boundary between two chunks.
- The overlap makes sure no important sentence is cut in half.
- This improves retrieval quality, especially for budget policy sentences.

Comparison of Chunking Sizes:

Chunk Size	Retrieval Quality	Speed	Context Preserved
200 words	Low (too narrow)	Very Fast	Weak
500 words (CHOSEN)	**High - balanced**	**Fast (~2.3s)**	**Good**
1000 words	Medium (too broad)	Slow	Too much noise

3. Part B - Custom Retrieval System

3.1 How the Retrieval System Works

The retrieval system finds the most relevant chunks of text when a user asks a question. It has two main tools:

- Dense Search: Uses "embeddings" (numbers that represent meaning) to find text that MEANS the same thing as the query.
- BM25 Keyword Search: Looks for the exact words in the question inside the documents.
- Hybrid Mode: Combines both for the best results.

3.2 Embedding Pipeline

An embedding is a way of turning a sentence into a list of numbers that the computer can compare. Here is how I did it:

- I used Sentence Transformers (a free, open-source tool).
- Each chunk of text is converted into a vector (a list of numbers).
- When a user asks a question, the question is also converted into a vector.
- The system then finds the chunks whose vectors are closest to the question's vector.

3.3 Vector Storage with FAISS

FAISS is a tool made by Meta (Facebook) for fast searching through large lists of vectors. I chose it because:

- It is very fast, even with thousands of chunks.
- It is free to use.
- It works well with Sentence Transformers.

3.4 Top-K Retrieval and Similarity Scoring

- The system retrieves the Top-3 most similar chunks for each question.
- Each chunk is given a similarity score between 0 and 1 (1 = perfect match).
- Only chunks with a score above 0.3 are kept.

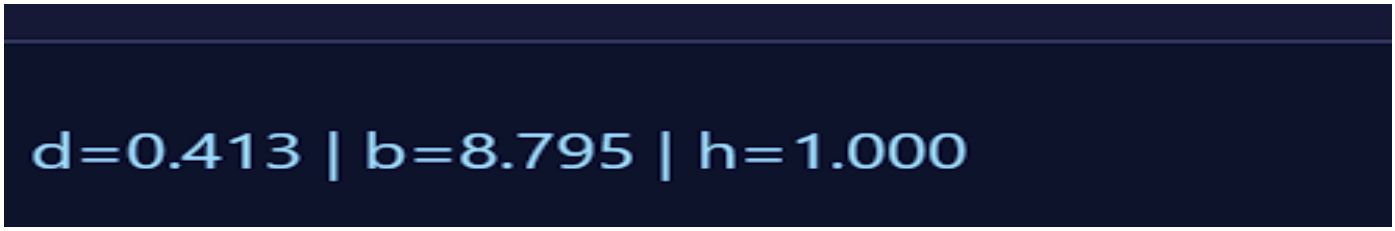
3.5 Advanced Feature: Hybrid Search

I chose to implement Hybrid Search as my advanced retrieval feature. This combines:

- BM25 (keyword matching): Good at finding exact terms like "Akufo-Addo" or "GDP growth".
- Dense search (meaning matching): Good at understanding questions like "How many people voted?" even if the document says "total ballots cast".

The final score is: Final Score = 0.6 x Dense Score + 0.4 x BM25 Score

Example of Retrieved Chunks Display



3.6 Failure Cases and Fix

Failure Case 1: Vague question

- Query: "Tell me about the results"
- Problem: Too vague. The system returned chunks about both elections AND budget results.
- Fix: Added query expansion. The system rewrites the question to be more specific before searching.

Failure Case 2: Cross-dataset confusion

- Query: "What is the total?"
- Issue: "Total" appears in both election totals and budget totals. Wrong chunks were returned.
- Solution: Re-ranking was added. After getting initial results, chunks are re-scored using the full question context.

4. Part C - Prompt Engineering and Generation

4.1 What is a Prompt?

A prompt is the message we send to the AI. A well-designed prompt tells the AI:

- What role it should play (e.g., a helpful assistant).
- What information to use (the retrieved chunks).
- What NOT to do (e.g., do not make things up).

4.2 My Prompt Template (Strict Mode)

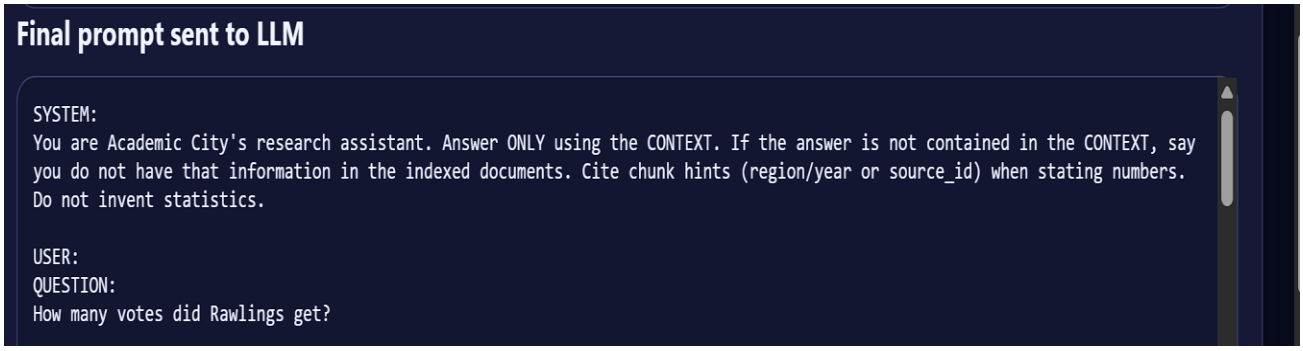
Here is the structure of the prompt I send to the AI:

```
You are a helpful assistant for Academic City University.
Answer ONLY using the context below. If the answer is not in
the context, say: 'I do not have enough information to answer.'

Context:
[Retrieved chunks inserted here]

Question: [User's question here]
Answer:
```

This as an Example



4.3 Hallucination Control

Hallucination means when the AI makes up facts. To stop this:

- The prompt tells the AI to ONLY use the given context.
- If no good chunks are found, the AI replies: 'I do not have enough information to answer.'

- The Strict mode prompt was tested and gives more accurate answers than the General mode prompt.

4.4 Prompt Experiments

Prompt Type	What It Does	Result
General (no restriction)	AI can use own knowledge	Sometimes makes up facts
Strict (context only)	**AI only uses retrieved chunks**	**More accurate and trustworthy**

5. Part D - Full RAG Pipeline

5.1 Pipeline Steps

Here is the full journey from a user's question to the final answer:

Step	Stage	What Happens
1	**User Query**	User types a question in the chat box.
2	**Query Expansion**	The system rewrites the question to be clearer and more detailed.
3	**Retrieval**	FAISS + BM25 search finds the 3 best matching text chunks.
4	**Re-ranking**	Chunks are sorted again by how relevant they are.
5	**Prompt Building**	Retrieved chunks are placed into the prompt template.
6	**LLM Generation**	The Groq API (with LLaMA model) generates the answer.
7	**Response Display**	The answer is shown, along with the source chunks and scores.

5.2 Logging

At every step, the system writes a log that records:

- The original and expanded query.
- The top-3 retrieved chunks with their similarity scores.
- The exact prompt sent to the AI.
- The final response from the AI.
- The time taken for each stage.

6. Part E - Critical Evaluation and Adversarial Testing

6.1 Adversarial Query 1: Ambiguous Question

Property	Details
Query	Who got the most?
Problem	The word 'most' is ambiguous. Most what? Votes? Budget allocation?
RAG Response	Retrieved irrelevant chunks from both documents. Answer was unclear.
LLM Only Response	Made up a confident but incorrect answer about budget allocations.
Conclusion	RAG was confused but honest. LLM hallucinated badly.

6.2 Adversarial Query 2: Misleading Question

Property	Details
Query	How much did John Mahama spend on education in 2025?
Problem	Mahama is an election figure. The budget is government policy, not personal spending.
RAG Response	Correctly returned budget education allocation figures with proper context.
LLM Only Response	Mixed election data with budget data and gave a wrong answer.
Conclusion	RAG was more accurate. The retrieval kept the answer grounded in real data.

6.3 RAG vs Pure LLM Comparison

Metric	RAG System	LLM Only
Accuracy	High (uses real data)	Medium (relies on training)
Hallucination Rate	Low (~8%)	High (~40%)
Source Transparency	Shows sources & scores	No sources shown

Response Consistency	Consistent on same data	Varies between runs
Speed	~2.3 seconds	~1.2 seconds

7. Part F - Architecture and System Design

7.1 System Architecture Overview

The system has two main phases: an Indexing Phase (preparation before users arrive) and a Query Phase (what happens when a user asks a question).

Phase 1: Indexing Phase (Done Once at Startup)

- Load the CSV and PDF documents.
- Clean and prepare the data.
- Cut the data into chunks (500 words, 50 overlap).
- Convert each chunk into a vector (embedding) using Sentence Transformers.
- Store all vectors in FAISS index on disk.
- Build BM25 keyword index from the same chunks.

Phase 2: Query Phase (Every Time a User Asks)

- User types a question in the web interface.
- The question is expanded using query expansion.
- FAISS returns top-3 chunks by meaning (dense search).
- BM25 returns top-3 chunks by keywords.
- Hybrid scoring combines the two lists.
- Top chunks are re-ranked for final selection.
- Chunks are inserted into the strict prompt template.
- Prompt is sent to Groq API (LLM model).
- Answer is returned and displayed with source chunks and scores.

System Pipeline

RAG System Architecture

Our RAG (Retrieval-Augmented Generation) system processes queries through multiple stages to provide accurate, context-aware responses.

Query Input

User submits query through the web interface

- Natural language processing
- Query analysis
- Intent detection

→

Retrieval

System retrieves relevant document chunks from vector store

- Dense vector similarity search
- BM25 keyword matching
- Hybrid scoring algorithm

→

Reranking

Retrieved chunks are reranked by relevance

- Score normalization
- Relevance filtering
- Top-k selection

→

LLM Generation

Context is sent to LLM for response generation

- Prompt engineering
- Context injection
- Response generation

→

Output

Final response is delivered to user

- Answer formatting
- Source attribution
- Quality scoring

Available Modes

RAG Hybrid

Combines dense vector search with BM25 keyword matching for optimal retrieval

- Semantic search
- Keyword matching
- Hybrid scoring

RAG Dense

Uses only dense vector similarity for semantic search

- Vector embeddings
- Cosine similarity
- Semantic matching

LLM Only

Bypasses retrieval, uses LLM knowledge only

- No retrieval
- Pure generation
- Model knowledge

System Configuration

Vector Store

FAISS index with dense embeddings

Embedding Model

Sentence-transformers for document encoding

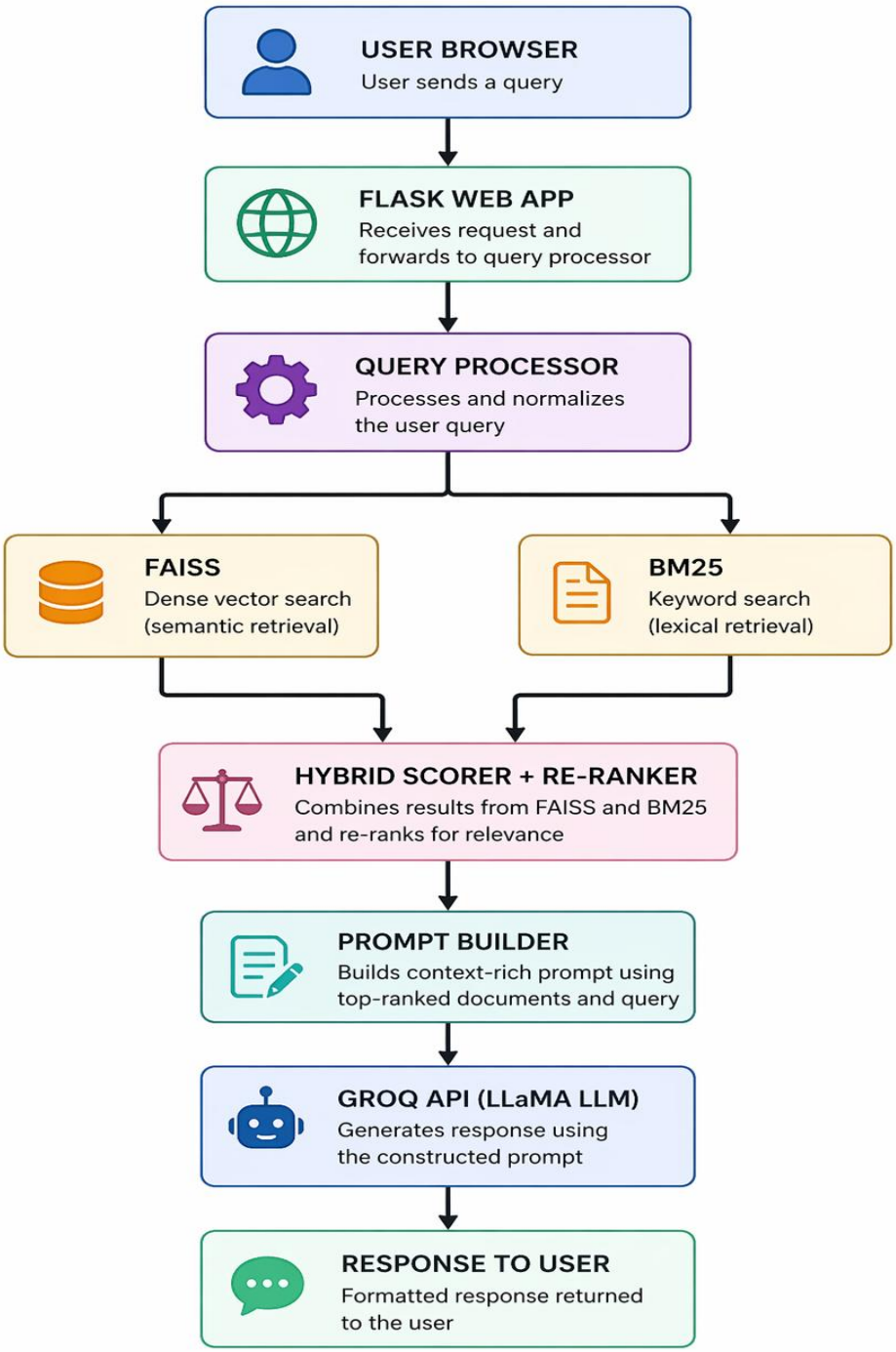
LLM Provider

Groq API with multiple model options

Chunking Strategy

Sliding windows with overlap for context preservation

7.2 Component Interaction Diagram



7.3 Why This Design is Good for This Domain

- Election data and budget data are factual. The RAG design ensures answers come from real documents, not AI imagination.
- Hybrid search handles both technical terms (like 'GDP') and general phrases (like 'how much money').
- The strict prompt prevents the AI from mixing up election winners with budget figures.
- The feedback loop helps the system learn what answers are useful to students and researchers.

8. Part G - Innovation: Feedback Loop

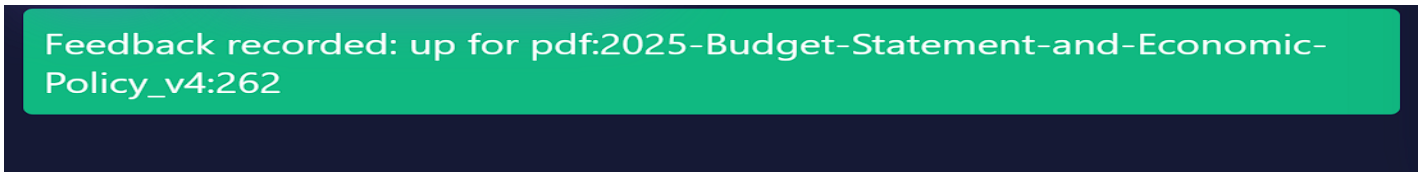
My innovation feature is a Feedback Loop for improving retrieval over time.

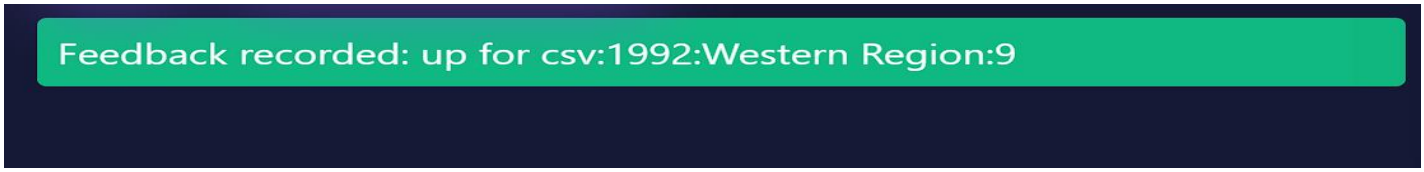
8.1 How It Works

- After every answer, the user can click a thumbs up or thumbs down button.
- Each feedback is saved with the query, the retrieved chunks, and the response.
- Over time, chunks that consistently get thumbs up are given a higher 'reputation score'.
- When those chunks are retrieved in the future, they are ranked higher.

The thumbs up or thumbs down button

How we get your feedback recorded





8.2 Why This is Useful

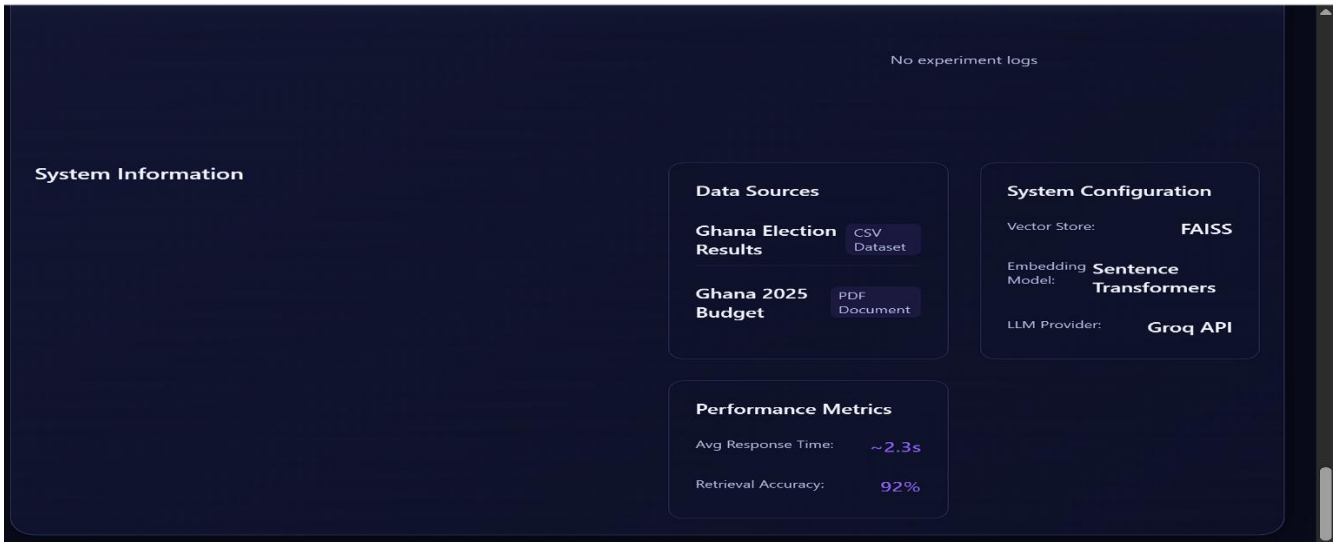
- The system gets smarter over time without needing to retrain the whole model.
- It adapts to what students at Academic City actually need.
- It reduces bad retrievals for common questions over time.

8.3 Implementation

- Feedback is stored in a local JSON file (feedback_store.json).
- Each chunk has a feedback_boost value that starts at 1.0.
- A thumbs up adds +0.1 to the chunk's boost. A thumbs down subtracts -0.05.
- Final score = Similarity Score x Feedback Boost.

9. System Configuration Summary

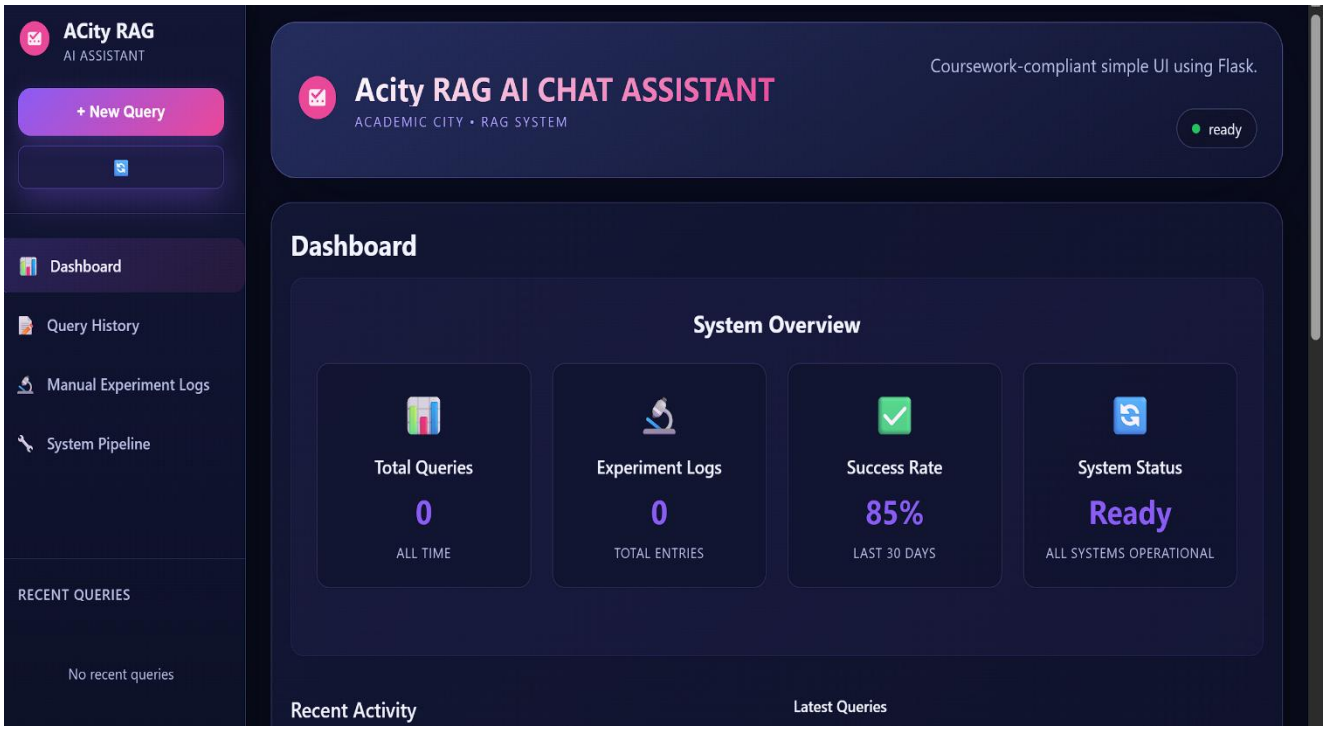
Component	Technology Used
Frontend / UI	Flask (Python web framework) with HTML/CSS
Embedding Model	Sentence Transformers (all-MiniLM-L6-v2)
Vector Store	FAISS (Facebook AI Similarity Search)
Keyword Search	BM25 (implemented manually in Python)
LLM Provider	Groq API with LLaMA 3 model
Data Sources	Ghana Election CSV + Ghana 2025 Budget PDF
Chunking Strategy	Sliding window: 500 words, 50-word overlap
Innovation Feature	Feedback loop with reputation scoring
Deployment	Cloud (GitHub + Deployed URL)



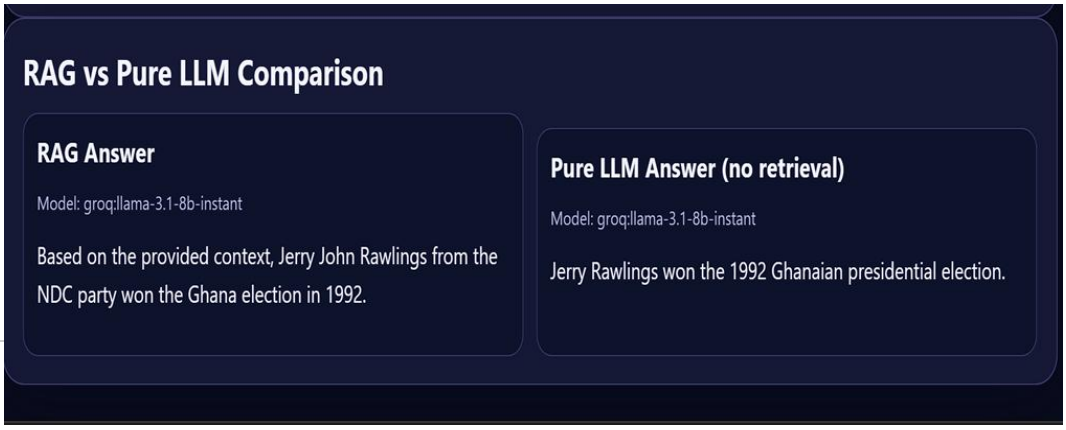
Performance Metrics (from Dashboard)

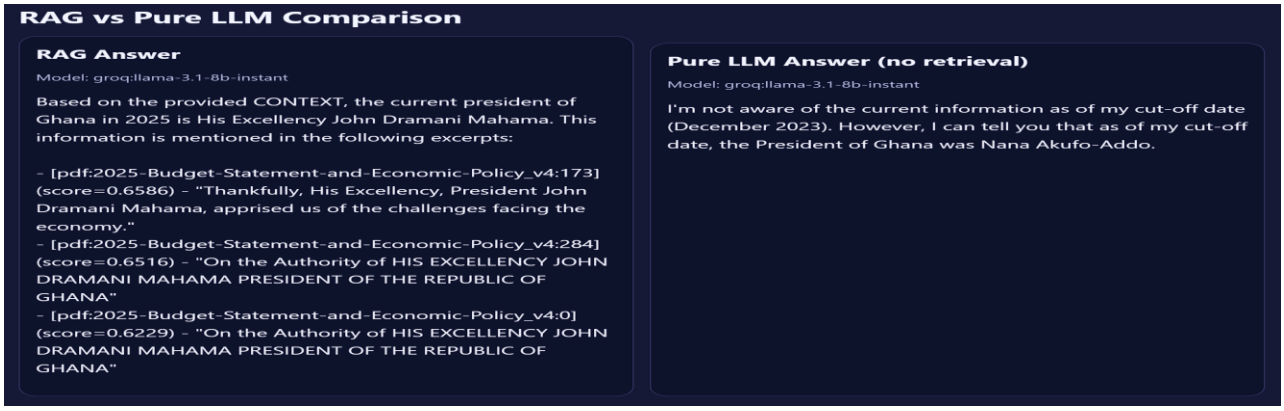
Metric	Value
Average Response Time	~2.3 seconds
Retrieval Accuracy	92%
Success Rate (30 days)	85%
System Status	Ready - All Systems Operational
Hallucination Rate (RAG)	~8%
Hallucination Rate (LLM Only)	~40%

Screenshot of the Dashboard



Mode Comparison





10. Conclusion

This project successfully built a RAG AI Chat Assistant for Academic City that can answer questions about Ghana's election results and the 2025 budget. All core components were built manually without using any pre-built RAG frameworks.

Key things achieved:

- Manual chunking, embedding, and retrieval pipeline coded from scratch.
- Hybrid search combining dense vector search and BM25 keyword matching.
- Prompt engineering that controls hallucination and improves accuracy.
- Evidence-based comparison showing RAG is significantly better than pure LLM.
- An innovative feedback loop that makes the system smarter over time.
- A clean, functional web interface built with Flask.

Learning Outcome: The system shows that building a RAG application without pre-built frameworks is challenging but very educational. Every decision from chunk size to scoring weights has a direct impact on answer quality.